Laboratorio 06

Tema: Django (Admin)

Profesor	Escuela	Asignatura
MSc. Ing. Lino Pinto Oppe	Escuela Profesional de Ingeniería de Sistemas	Programación Web Semestre: III Código: 1702122

Laboratorio	Tema	Duración
06	Django (Admin)	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	21/10/2026	28/10/20206

Índice

1. Competencias del curso **2**

2. Resultado del estudiante **2**

3. Equipos, materiales y temas **2**

4. Directorio de trabajo **3**

5. Marco teórico **5**
5.1. Django 5
5.2. Crear un directorio para el entorno virtual de Django 5
5.3. Crear entorno virtual en un directorio 5
5.4. Activando entorno virtual 6
5.5. Desactivando entorno virtual 6
5.6. Instalando Django dentro del entorno virtual 6

6. Ejercicios **8**

7. Ejemplo de una WebApp: Django Administrador **8**
7.1. Modelo de datos: Diagrama E-R 8
7.2. Diccionario de datos 8
7.3. Activando Entorno Virtual 10
7.4. Instalando Django 10
7.5. Creando un proyecto Django 10
7.6. Creando aplicaciones Django 10
7.7. Creando Modelos 10
7.8. Registrando Modelos 16
7.9. Configurar el Proyecto Django 16
7.10. Configurar ubicación de la aplicación 17
7.11. Ejecutar Migraciones 17

7.12. Crear superusuario 17
7.13. Crear migraciones 18
7.14. Ejecutar proyecto Django 18
7.15. Panel de administración 18
7.16. .gitignore 19
7.17. requirements.txt 20
7.18. Estructura del Proyecto Django 21

8. Tarea **23**

9. Pregunta **23**

10. Entregables **24**

11. Rúbricas **24**
11.1. Rúbrica para entregable Informe 24
11.2. Rúbrica para el contenido del Informe y demostración 25

12. Referencias **27**

1. Competencias del curso

- General: C.c. Diseña responsablemente aplicaciones web, sus componentes o procesos para satisfacer necesidades dentro de restricciones realistas: económicas, medio ambientales, sociales, políticas, éticas, de salud, de seguridad, manufacturación y sostenibilidad.
- Específica: C.m. Construye responsablemente soluciones con tecnología web siguiendo un proceso adecuado llevando a cabo las pruebas ajustada a los recursos disponibles del cliente.
- Específica: C.p. Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas del desarrollo web necesarias para la construcción de aplicaciones web e implementación de estos sistemas en una organización.

2. Resultado del estudiante

- RE. 2 La capacidad de aplicar diseño de ingeniería para producir soluciones a problemas y diseñar sistemas, componentes o procesos para satisfacer necesidades específicas dentro de consideraciones realistas en los aspectos de salud pública, seguridad y bienestar; factores globales, culturales, sociales, económicos y ambientales.
- RE. 8 La capacidad de crear, seleccionar y utilizar técnicas, habilidades, recursos y herramientas modernas de ingeniería y tecnologías de la información, incluyendo la predicción y el modelamiento, con una comprensión de las limitaciones.

3. Equipos, materiales y temas

- Sistema Operativo (GNU/Linux de preferencia).
- GNU Vim.
- Python 3.
- Git.

- Cuenta en GitHub con el correo institucional.
- Entorno virtual.
- Django 4.

4. Directorio de trabajo

- Cree su directorio de trabajo.
- Luego, diríjase a este directorio, para clonar o sincronizar su repositorio y continuar sus practicas.

Listing 1: Creando directorio de trabajo local

```
$ mkdir -p $HOME/rescobedoq/
```

Listing 2: Dirijíendonos al directorio de trabajo local

```
$ cd $HOME/rescobedoq/
```

- Clonando :

Listing 3: Clonando repositorio GitHub

```
$ git clone https://github.com/rescobedoq/pw2-lab-24a.git
```

- Sincronizando :

Listing 4: Sincronizando repositorio local con repositorio GitHub

```
$ cd $HOME/rescobedoq/pw2-lab-24a
$ git pull
```

Listing 5: Creando directorio para este laboratorio

```
$ mkdir -p $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/
```

- Siempre evalue utilizar el archivo .gitignore para no considerar algunos archivo innecesarios sobre todo para el repositorio GitHub.
- Pueden haber varios de estos archivos y estar ubicados estratégicamente; por ejemplo sólo para un laboratorio en particular.

Listing 6: Creando .gitignore

```
$ vim $HOME/rescobedoq/pw2-lab-24a/lab05/.gitignore
```

Listing 7: Ejemplo de .gitignore

```
my_env/bin/*
my_env/lib/*
my_env/src/__pycache__/*
*.pyc
```

- Estudie el archivo .gitignore del proyecto Library :
- https://github.com/mdn/django-locallibrary-tutorial/blob/main/.gitignore

5. Marco teórico

5.1. Django

- Django es un framework web Python con el cuál el desarrollo de sitios web son rápidos, seguros y sobre todo fáciles de mantener.
- Django se ocupa de gran parte de las molestias del desarrollo web, por lo que puede concentrarse en escribir su aplicación sin necesidad de reinventar la rueda.
- Es software libre, tiene una comunidad próspera y activa, excelente documentación y muchas opciones de soporte gratuito y de pago.

5.2. Crear un directorio para el entorno virtual de Django

- Para crear un ambiente elija en qué directorio se va crear el entorno virtual.
- No envíe estos entornos virtuales al GitHub, omítalos en .gitignore. Pero no olvide tener requirements.txt en el directorio raíz del Proyecto.

Listing 8: Creando directorio para entorno virtual en Unix

```
$ mkdir -p $HOME/rescobedoq/pw2-lab-24a/lab05/my_env
```

Listing 9: Creando directorio para entorno virtual

```
$ mkdir -p $HOME/rescobedoq/pw2-lab-24a/lab05/my_env
```

5.3. Crear entorno virtual en un directorio

- En este directorio crear un entorno virtual ejecutando el siguiente comando:

Listing 10: Creando entorno virtual en GNU/Linux

```
$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/my_env
$ virtualenv -p python3 .
```

Listing 11: Creando entorno virtual en MS Windows

```
C:\>python -m venv c:\rescobedoq\pw2-lab-24a\lab05\my_env
```

Listing 12: Creando entorno virtual en MacOSX

```
$ python -m venv $HOME/rescobedoq/pw2-lab-24a/lab05/my_env
```

5.4. Activando entorno virtual

- En el directorio de trabajo active el entorno virtual ejecutando el script activate.
- Sea cual sea nuestro sistema operativo sabremos que el entorno virtual se ha activado porque su nombre aparece entre paréntesis delante del promt.

Listing 13: Activando entorno virtual en GNU/Linux

```
$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/
$ source ./../my_env/bin/activate
(my_env) user@localhost:$
```

Listing 14: Activando entorno virtual en MS Windows

```
C:\rescobedoq\pw2-lab-24a\lab05\my_env\scripts\activate.bat
```

Listing 15: Activando entorno virtual en MacOSX

```
$ source $HOME/rescobedoq/pw2-lab-24a/lab05/my_env/bin/activate
```

5.5. Desactivando entorno virtual

- El comando para desactivar el entorno virtual es idéntico para Windows, MacOSX y Linux:

Listing 16: Desactivando entorno virtual

```
$ deactivate
```

5.6. Instalando Django dentro del entorno virtual

- Siempre hay que estar consientes de los paquetes de nuestro entorno virtual.

Listing 17: Mostrando paquetes instalados en el entorno virtual

```
(my_env) user@localhost:$ pip list
Package    Version
---------- ----------
pip        22.0.4
setuptools 62.1.0
wheel      0.37.1
```

- Instalamos Django con pip:

Listing 18: Instando Django

```
(my_env) user@localhost:$ pip install Django
```

- Volvemos a listar los paquetes instalados:

Listing 19: Mostrando Django instalado en el entorno virtual

```
(my_env) user@localhost:$ pip list
Package    Version
---------- ---------------
asgiref    3.5.2
Django     4.0.5
pip        22.0.4
setuptools 62.1.0
sqlparse   0.4.2
wheel      0.37.1
```

- **Importante**: Como el entorno virtual no se clona, por temas de espacio. Es necesario sacarle un backup en un arquivo requirements.txt

Listing 20: Creando requirements.txt

```
(my_env) user@localhost:$ pip freeze > requirements.txt
```

- **Importante**: El arquivo requirements.txt, servirá para volver a construir el entorno virtual, preservando los paquetes con sus versiones.

Listing 21: Instalando desde requirements.txt

```
(my_env) user@localhost:$ pip install -r requirements.txt
```

6. Ejercicios

- Cree la aplicación Library paso a paso desde la siguiente url:
- https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Tutorial_local_library_website

7. Ejemplo de una WebApp: Django Administrador

- Django Parte I : Django Admin
- El siguiente ejemplo es parte de una implementación para una pequeña aplicación web para permitir inscripciones de estudiantes a cursos de laboratorio.

7.1. Modelo de datos: Diagrama E-R

- El modelo de datos de esta versión esta conformado por 5 tablas.

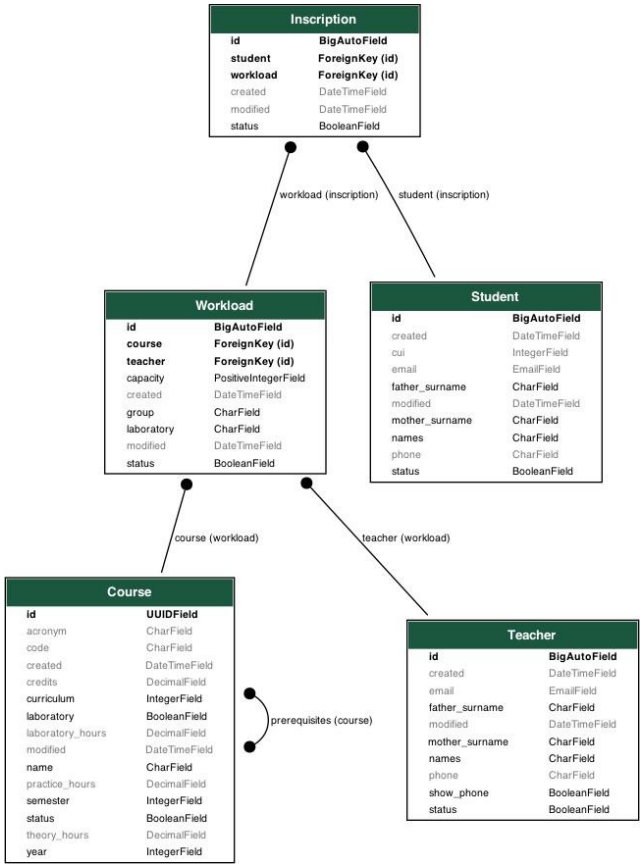


Figura 1: Modelo de datos Enrollments 1.0

7.2. Diccionario de datos

- Course : El modelo Course registra los datos de los cursos que se imparten.

Tabla 1: Course

id	2d40d211b4104a87b7e097bff85619b4
acronym	PW2
code	1702122
created	2022-08-16 14:59:49.485875
credits	4
curriculum	2
laboratory	1
laboratory_hours	4
modified	2022-08-16 14:59:49.485875
name	PROGRAMACION WEB 2
practice_hours	0
semester	3
status	1
theory_hours	2
year	1

- Teacher : El modelo Teacher registra los datos de los profesores.

Tabla 2: Teacher

id	1
created	2022-08-16 15:13:08.186656
email	rescobedoq@unsa.edu.pe
father_surname	ESCOBEDO
modified	2022-08-16 15:45:01.141900
mother_surname	QUISPE
names	RICHART SMITH
phone	
show_phone	0
status	1

- Workload : El modelo Workload registra las asignaciones de cursos a los profesores.
- Student : El modelo Student registra a todos los estudiantes matriculados.
- Inscription : El modelo Inscription registra las inscripciones de estudiantes en determinados grupos de laboratorio.

7.3. Activando Entorno Virtual

- Para activar el entorno virtual, ejecute el script activate.

Listing 22: Activando entorno virtual en GNU/Linux

```
$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/
$ source ./../my_env/bin/activate
(my_env) user@localhost:$
```

7.4. Instalando Django

- Para instalar Django utilice el comando pip install.

Listing 23: Instando Django en el entorno virtual

```
(my_env) user@localhost:$ pip install Django
```

7.5. Creando un proyecto Django

- Para crear el proyecto Django, ejecute el siguiente comando:

Listing 24: Creando Proyecto Django

```
(my_env) user@localhost:$ django-admin startproject MyDjangoProject
```

7.6. Creando aplicaciones Django

- Crear directorio para alojar las aplicaciones:

Listing 25: Ubicándose en la raiz del proyecto

```
(my_env) user@localhost:$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/MyDjangoProject
```

Listing 26: Creando aplicaciones Django

```
(my_env) user@localhost:$ mkdir MyWebApps
(my_env) user@localhost:$ cd MyWebApps
(my_env) user@localhost:$ django-admin startapp MyFirstApplication
```

7.7. Creando Modelos

- Hacer obsoleto models.py para crear modelos en archivos individuales:

```
(my_env) user@localhost:$ cd MyFirstApplication
(my_env) user@localhost:$ mv models.py models.py.deprecated
(my_env) user@localhost:$ mkdir models
```

Listing 27: Ubicándose en la raiz del proyecto

```
(my_env) user@localhost:$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/MyDjangoProject
```

- Crear el modelo Course:

Listing 28: Creando el archivo Course.py

```
$ vim MyWebApps/MyFirstApplication/models/Course.py
```

Listing 29: models/Course.py

```
1  from django.db import models
2
3  from django.utils.translation import gettext_lazy as _
4
5  import uuid
6
7  class Course(models.Model):
8      id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
9      CURRICULUMS = [
10         (0, 'Sin Plan'),
11         (2017, 'Plan 2017'),
12         (2023, 'Plan 2023'),
13     ]
14     curriculum = models.IntegerField(null=False, choices=CURRICULUMS, default=2017)
15     YEARS = [
16         (0, 'Sin ñao'),
17         (1, '1er ñao'),
18         (2, '2do ñao'),
19         (3, '3er ñao'),
20         (4, '4to ñao'),
21         (5, '5to ñao'),
22         (6, '6to ñao'),
23         (7, '7mo ñao'),
24     ]
25     year = models.IntegerField(null=False, choices=YEARS, default=0)
26     SEMESTERS = [
27         (0, 'Sin semestre'),
28         (1, 'I semestre'),
29         (2, 'II semestre'),
30         (3, 'III semestre'),
31         (4, 'IV semestre'),
32         (5, 'V semestre'),
33         (6, 'VI semestre'),
34         (7, 'VII semestre'),
35         (8, 'VIII semestre'),
```

```
        (9, 'IX semestre'),
        (10, 'X semestre'),
    ]
    semester = models.IntegerField(null=False, choices=SEMESTERS, default=0)
    code = models.CharField(unique=True, null=True, blank=True, max_length=25)
    name = models.CharField(null=False, blank=False, max_length=255)
    acronym = models.CharField(null=True, blank=True, max_length=25)
    #enrolled = models.CharField(max_length=100)
    credits = models.DecimalField(null=True, blank=True, max_digits=4, decimal_places=2)
    theory_hours = models.DecimalField(null=True, blank=True, max_digits=5, decimal_places=2)
    practice_hours = models.DecimalField(null=True, blank=True, max_digits=5,
        decimal_places=2)
    laboratory_hours = models.DecimalField(null=True, blank=True, max_digits=5,
        decimal_places=2)
    laboratory = models.BooleanField(default=True, null=False)
    status = models.BooleanField(default=True, null=False)
    created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
    modified = models.DateTimeField(null=False, auto_now=True)
    #prerequisites = models.ManyToManyField(Course)
    prerequisites = models.ManyToManyField('self', blank=True, symmetrical=False)

    class Meta:
        ordering = ['curriculum', 'year', 'semester', 'code', 'name']

    def save(self, *args, **kwargs):
        self.name = self.name.upper()
        if self.acronym is not None:
            self.acronym = self.acronym.upper()
        return super(Course, self).save(*args, **kwargs)

    def __str__(self):
        return "%s %s %s %s %s" % (self.curriculum, self.year, self.semester, self.code,
            self. name)
```

- Crear el modelo Teacher:

Listing 30: Creando el archivo Teacher.py

```
$ vim MyWebApps/MyFirstApplication/models/Teacher.py
```

Listing 31: models/Teacher.py

```
from django.db import models

from django.utils.translation import gettext_lazy as _

import uuid

class Teacher(models.Model):
    names = models.CharField(null=False, blank=False, max_length=155)
    father_surname = models.CharField(null=False, blank=False, max_length=155)
    mother_surname = models.CharField(null=False, blank=False, max_length=155)
    email = models.EmailField(unique=True, null=True, blank=True, max_length=255)
```

```
12      phone = models.CharField(null=True, blank=True, max_length=255)
13      show_phone = models.BooleanField(default=False, null=False)
14      status = models.BooleanField(default=True, null=False)
15      created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
16      modified = models.DateTimeField(null=False, auto_now=True)
17
18      class Meta:
19          ordering = ['names', 'father_surname', 'mother_surname']
20
21      def save(self, *args, **kwargs):
22          self.names = self.names.upper()
23          self.father_surname = self.father_surname.upper()
24          self.mother_surname = self.mother_surname.upper()
25          return super(Teacher, self).save(*args, **kwargs)
26
27      def __str__(self):
28          return "%s %s %s" % (self.names, self.father_surname, self.mother_surname)
```

- Crear el modelo Workload:

Listing 32: Creando el archivo Workload.py

```
$ vim MyWebApps/MyFirstApplication/models/Workload.py
```

Listing 33: models/Workload.py

```
1  from django.db import models
2
3  from django.utils.translation import gettext_lazy as _
4
5  import uuid
6
7  from .Course import Course
8  from .Teacher import Teacher
9
10 class Workload(models.Model):
11     course = models.ForeignKey(Course, on_delete=models.CASCADE)
12     GROUPS = [
13         ('A', 'A'),
14         ('B', 'B'),
15         ('C', 'C'),
16         ('D', 'D'),
17         ('E', 'E'),
18         ('F', 'F'),
19     ]
20     group = models.CharField(null=False, max_length=1, choices=GROUPS, default='A')
21     LABORATORIES = [
22         ('lab01', 'Laboratorio 01'),
23         ('lab02', 'Laboratorio 02'),
24         ('lab03', 'Laboratorio 03'),
25         ('lab04', 'Laboratorio 04'),
26         ('lab05', 'Laboratorio 05'),
27         ('lab06', 'Laboratorio 06'),
```

```
        ('lab07', 'Laboratorio 07'),
        ('lab08', 'Laboratorio 08'),
    ]
    laboratory = models.CharField(null=False, max_length=5, choices=LABORATORIES,
        default='lab01')
    capacity = models.PositiveIntegerField(null=False, default=20)
    teacher = models.ForeignKey(Teacher, on_delete=models.CASCADE)
    status = models.BooleanField(default=True, null=False)
    created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
    modified = models.DateTimeField(null=False, auto_now=True)

    class Meta:
        ordering = ['course', 'group', 'laboratory', 'capacity', 'teacher']
        constraints = [
            models.UniqueConstraint(fields=['course', 'group'], name='unique_workload')]

    def __str__(self):
        return "%s %s %s %s %s" % (self.course, self.group, self.laboratory, self.capacity,
            self.teacher)
```

- Crear el modelo Student:

Listing 34: Creando el archivo Student.py

```
$ vim MyWebApps/MyFirstApplication/models/Student.py
```

Listing 35: models/Student.py

```
from django.db import models

from django.utils.translation import gettext_lazy as _

import uuid

class Student(models.Model):
    cui = models.IntegerField(unique=True, null=True, blank=True)
    names = models.CharField(null=False, blank=False, max_length=155)
    father_surname = models.CharField(null=False, blank=False, max_length=155)
    mother_surname = models.CharField(null=False, blank=False, max_length=155)
    email = models.EmailField(unique=True, null=True, blank=True, max_length=255)
    phone = models.CharField(null=True, blank=True, max_length=255)
    status = models.BooleanField(default=True, null=False)
    created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
    modified = models.DateTimeField(null=False, auto_now=True)

    class Meta:
        ordering = ['cui', 'names', 'father_surname', 'mother_surname']

    def save(self, *args, **kwargs):
        self.names = self.names.upper()
        self.father_surname = self.father_surname.upper()
        self.mother_surname = self.mother_surname.upper()
        return super(Student, self).save(*args, **kwargs)
```

```
26
27      def __str__(self):
28          return "%s %s %s %s" % (self.cui, self.names, self.father_surname,
                self.mother_surname)
```

- Crear el modelo Inscription:

Listing 36: Creando el archivo Inscription.py

```
$ vim MyWebApps/MyFirstApplication/models/Inscription.py
```

Listing 37: models/Inscription.py

```
1  from django.db import models
2
3  from django.db.models.constraints import UniqueConstraint
4
5  from django.core.exceptions import ValidationError, NON_FIELD_ERRORS
6  from django.utils.translation import gettext_lazy as _
7
8  import uuid
9
10 from .Workload import Workload
11 from .Student import Student
12
13 def validate_even(value):
14     if value == 1:
15         raise ValidationError(
16         _('%(value)s is 1'),
17         params={'value': value},
18     )
19
20 class Inscription(models.Model):
21     workload = models.ForeignKey(Workload, on_delete=models.CASCADE)
22     student = models.ForeignKey(Student, on_delete=models.CASCADE, validators=[validate_even])
23     status = models.BooleanField(default=True, null=False)
24     created = models.DateTimeField(editable=False, null=False, auto_now_add=True)
25     modified = models.DateTimeField(null=False, auto_now=True)
26
27     class Meta:
28         ordering = ['workload', 'student', 'created']
29         constraints = [
30             models.UniqueConstraint(fields=['workload', 'student'], name='unique_inscription')]
31
32     def clean(self):
33         if (self.workload.id == 3 and self.student.id == 2):
34             raise ValidationError("No se puede inscribir workload=3 student=2")
35         super(Inscription, self).clean()
36
37     #def save(self, *args, **kwargs):
38     #    if self.workload == 3 and self.student == 2:
39     #        raise ValueError("No se puede inscribir workload=3 student=2")
40     #    return super(Inscription, self).save(*args, **kwargs)
```

```
41
42      def save(self, *args, **kwargs):
43          #if (self.workload == 3 and self.student == 2):
44          self.clean()
45          return super(Inscription, self).save(*args, **kwargs)
46
47      def __str__(self):
48          return "%s %s" % (self.workload, self.student)
```

7.8. Registrando Modelos

- Registraremos los modelos:

Listing 38: Editando el archivo admin.py

```
$ vim MyWebApps/MyFirstApplication/admin.py
```

Listing 39: admin.py

```
1  from django.contrib import admin
2
3  from .models.Course import Course
4  from .models.Teacher import Teacher
5  from .models.Workload import Workload
6  from .models.Student import Student
7  from .models.Inscription import Inscription
8
9  admin.site.register(Course)
10 admin.site.register(Teacher)
11 admin.site.register(Workload)
12 admin.site.register(Student)
13 admin.site.register(Inscription)
```

7.9. Configurar el Proyecto Django

- Añadir la aplicación como una aplicación instalada:

Listing 40: Editando el archivo settings.py

```
$ vim MyDjangoProject/settings.py
```

Listing 41: settings.py

```
#...
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
```

```
        'django.contrib.contenttypes',
        'django.contrib.sessions',
        'django.contrib.messages',
        'django.contrib.staticfiles',
        'MyWebApps.MyFirstApplication'
]
#...
# Internationalization
#...
LANGUAGE_CODE = 'es-PE'
#...
TIME_ZONE = 'America/Lima'
#...
```

7.10. Configurar ubicación de la aplicación

- Agregar el directorio contenedor al nombre de la aplicacion:

Listing 42: Editando el archivo apps.py

```
$ vim MyWebApps/MyFirstApplication/apps.py
```

Listing 43: apps.py

```
from django.apps import AppConfig

class Aplicacion1Config(AppConfig):
        default_auto_field = 'django.db.models.BigAutoField'
        name = 'MyWebApps.MyFirstApplication'
```

7.11. Ejecutar Migraciones

- Realizar la primera migración:

Listing 44: Ubicándose en la raiz del proyecto

```
$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/MyDjangoProject
```

```
(my_env) user@localhost:$ python manage.py migrate
```

7.12. Crear superusuario

- Crear el super usuario para poder ingresar al panel de administración:

```
(my_env) user@localhost:$ python manage.py createsuperuser
```

- **Importante**: Para todos sus proyectos Django utilice.
- Usuario: admin
- Contraseña: 1234

```
Nombre de usuario (leave blank to use richart): admin
Direccion de correo electronico:
Password: 1234
Password (again): 1234
Esta ñcontrasea es demasiado corta. Debe contener al menos 8 caracteres.
Esta ñcontrasea es demasiado comun.
Esta ñcontrasea es completamente numerica.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

7.13. Crear migraciones

- Crear las migraciones. Las migraciones iniciales crearan los modelos.

```
(my_env) user@localhost:$ python manage.py makemigrations
```

- Realizar una nueva migración para que los cambios se efectuen.

```
(my_env) user@localhost:$ python manage.py migrate
```

7.14. Ejecutar proyecto Django

- Ejecutar el servidor:

```
(my_env) user@localhost:$ python manage.py runserver
```

- Acceda desde un navegador web a la dirección : http://127.0.0.1:8000/admin/
- Pruebe los CRUDs en cada una de las tablas.

7.15. Panel de administración



Figura 2: Django Admin tables

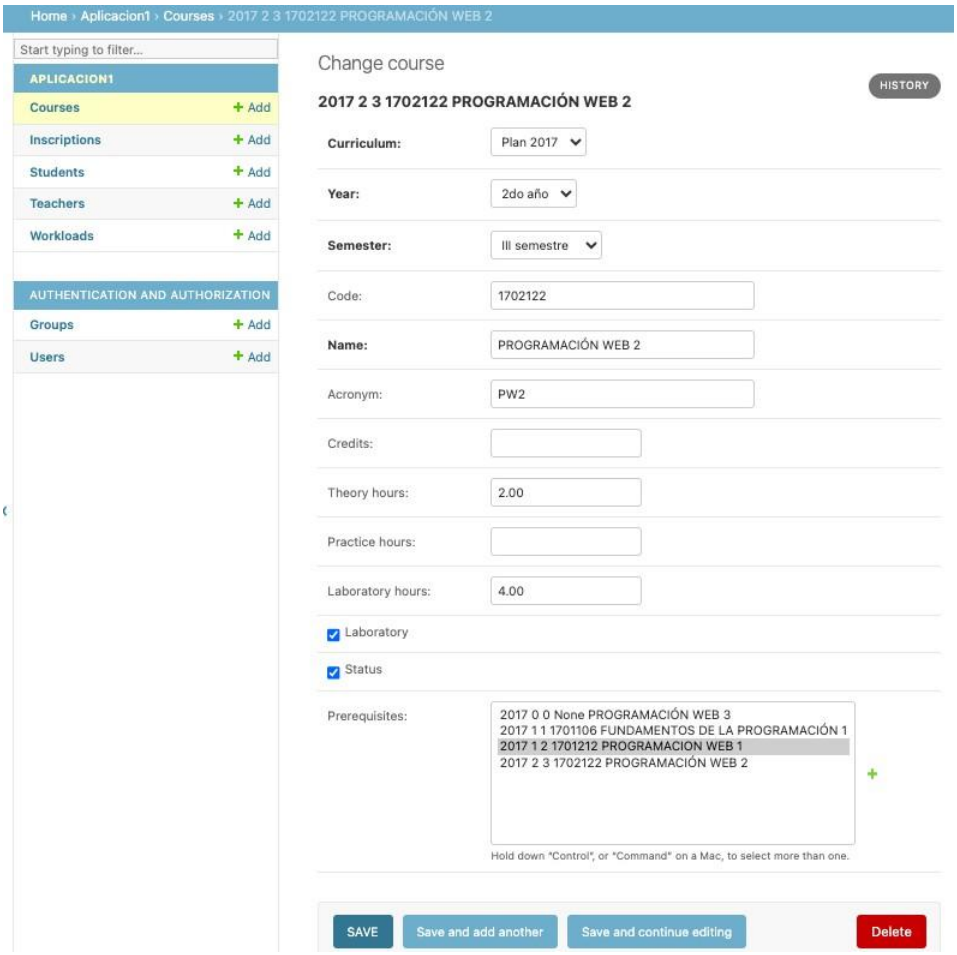


Figura 3: Django Admin Agregando un curso

7.16. .gitignore

- **Importante**: Considere utilizar otro archivo .gitignore ubicado esta vez en la raiz de su proyecto Django, sobre todo para no considerar algunos archivo innecesarios en el repositorio GitHub.

Listing 45: Ubicándose en la raiz del proyecto

```
$ cd $HOME/rescobedoq/pw2-lab-24a/lab05/exercises/MyDjangoProject
```

Listing 46: Creando .gitignore para el Proyecto Django

```
$ vim .gitignore
```

Listing 47: Ejemplo de .gitignore

```
my_env/bin/*
my_env/lib/*
```

```
my_env/src/__pycache__/*
*.pyc
__pycache_
```

- Estudie el archivo .gitignore del proyecto Library :
- https://github.com/mdn/django-locallibrary-tutorial/blob/main/.gitignore

7.17. requirements.txt

- **Importante**: Como el entorno virtual no se clona, por temas de espacio. Es necesario sacarle un backup en un arquivo requirements.txt ubicado también en la raiz del proyecto.

Listing 48: Creando requirements.txt

```
(my_env) user@localhost:$ pip freeze > requirements.txt
```

- **Importante**: El arquivo requirements.txt, servirá para volver a construir el entorno virtual, preservando los paquetes con sus versiones.

Listing 49: Instalando desde requirements.txt

```
(my_env) user@localhost:$ pip install -r requirements.txt
```

7.18. Estructura del Proyecto Django

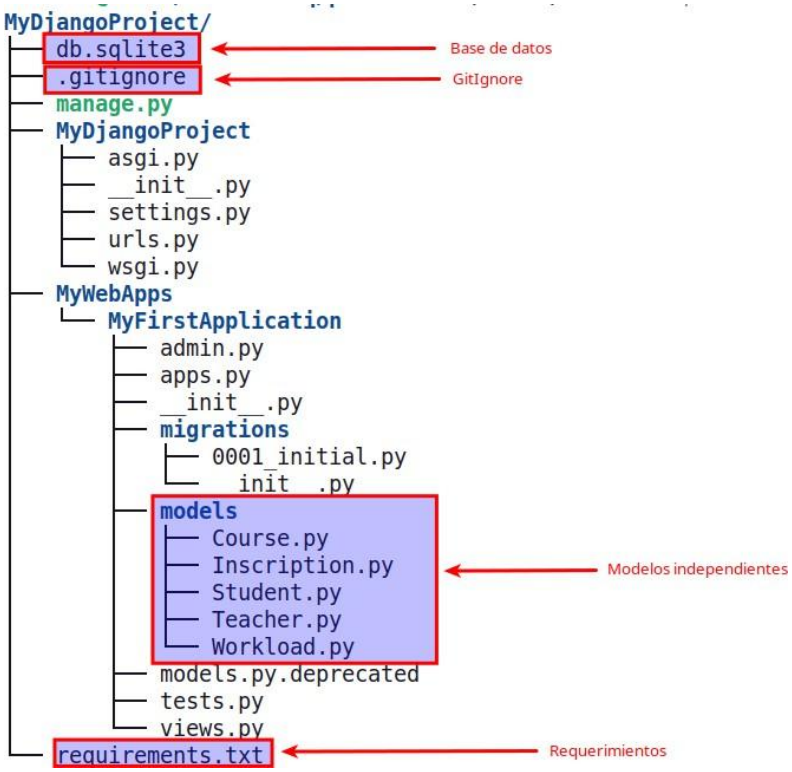
- Para obtener la estructura del Proyecto Django con el comando tree:

Listing 50: Instalando desde requirements.txt

```
(my_env) user@localhost:$ tree -a -L 4 --gitignore MyDjangoProject/
```

```
MyDjangoProject/
|--- db.sqlite3
|--- .gitignore
|--- manage.py
|--- MyDjangoProject
|    |--- asgi.py
|    |--- __init__.py
|    |--- settings.py
|    |--- urls.py
|    |--- wsgi.py
|--- MyWebApps
|    |--- MyFirstApplication
|         |--- admin.py
|         |--- apps.py
|         |--- __init__.py
|         |--- migrations
|         |    |--- 0001_initial.py
|         |    |--- __init__.py
|         |--- models
|         |    |--- Course.py
|         |    |--- Inscription.py
|         |    |--- Student.py
|         |    |--- Teacher.py
|         |    |--- Workload.py
|         |--- models.py.deprecated
|         |--- tests.py
|         |--- views.py
|--- requirements.txt
```

- Verifique archivos en estructura del Proyecto Django:



8. Tarea

- Elabore un primer informe grupal, con el modelo de datos de una aplicación que desarrollará durante este semestre.
- Utilicen todas las recomendaciones encontradas en la aplicación library.
- Acuerdos :
  - Los grupos pueden estar conformado por 1 a 4 integrantes.
  - Sólo se presenta un informe grupal.
  - Sólo se revisa un repositorio. (Avisar al profesor, en el cuál esté el informe grupal).
  - Todos los integrantes deben tener una copia del laboratorio e informe en su repositorio privado también.
  - Todos los integrantes deben pertenecer al mismo grupo de laboratorio y en el mismo horario.
  - El docente preguntará en cualquier momento a cualquier integrante sobre el proyecto, modelo de datos, codigo fuente, avance.

9. Pregunta

- Por cada integrante del equipo, resalte un aprendizaje que adquirió al momento de estudiar esta primera parte de Django (Admin). No se reprima de ser detallista. Coloque su nombre entre parentesis para saber que es su aporte.

10. Entregables

- El informe debe tener un enlace al directorio específico del laboratorio en su repositorio GitHub privado donde esté todo el código fuente y otros que sean necesarios. Evitar la presencia de archivos: binarios, objetos, archivos temporales, cache, librerias, entornos virtuales. Si hay configuraciones particulares puede incluir archivos de especificación como: requirements.txt, o leeme.txt.
- No olvide que el profesor debe ser siempre colaborador a su repositorio (Usuario del profesor **@rescobedoq**).
- Para ser considerado con la calificación de máxima nota, el informe debe estar elabórado en LaTeX
- Usted debe describir sólo los **commits** más importantes que marcaron hitos en su trabajo, adjutando capturas de pantalla, del commit, del código fuente, de sus ejecuciones y pruebas.
- En el informe siempre se debe explicar las imágenes (código fuente, capturas de pantalla, commits, ejecuciones, pruebas, etc.) con descripciones puntuales pero precisas.
- Partes de entrega:
  - Modelo de datos. (Diagrama Entidad-Relación)
  - Modelos Python. (En archivos independientes)
  - Implementación del Django Administrador. (CRUD para todas las tablas)
  - Considerar: atributos adecuados, relaciones necesarias, visualización de datos adecuadas, restricciones en el modelo importantes.

11. Rúbricas

11.1. Rúbrica para entregable Informe

Tabla 3: Rúbrica para tipo de Informe

Informe		Cumple	No cumple
LaTeX	El informe está en formato PDF desde LaTeX, con un formato limpio (buena presentación) y facil de leer.	20	0
MarkDown	El informe está en formato PDF desde MarkDown README.md, con un formato limpio (buena presentación) y facil de leer.	17	0
MS Word	El informe está en formato PDF desde plantilla MS Word, con un formato limpio (buena presentación) y facil de leer.	15	0
Observaciones	Por cada observación se le descontará puntos.	-	-

11.2. Rúbrica para el contenido del Informe y demostración

- El alumno deberá marcar o dejar en blanco en las celdas de la columna **Checklist**, deacuerdo a si cumplió o no con el ítem correspondiente.
- Si un alumno supera la fecha de entrega, su calificación siempre será sobre la nota mínima aprobada, siempre y cuando cumpla con todos lo items.
- El alumno debe autocalificarse en la columna **Estudiante** de acuerdo a la tabla de calificación de niveles de desempeño:

Tabla 4: Niveles de desempeño

	Nivel			
Puntos	Insatisfactorio 25 %	En Proceso 50 %	Satisfactorio 75 %	Sobresaliente 100 %
2.0	0.5	1.0	1.5	2.0
4.0	1.0	2.0	3.0	4.0

Tabla 5: Rúbrica para contenido del Informe y demostración

Contenido y demostración		Puntos	Checklist	Estudiante	Profesor
1. GitHub	Hay enlace URL activo del directorio para el laboratorio hacia su repositorio GitHub con código fuente terminado y fácil de revisar.	2			
2. Commits	Hay capturas de pantalla de los commits más importantes con sus explicaciones detalladas. (El profesor puede preguntar para refrendar calificación).	4			
3. Código fuente	Hay porciones de código fuente importantes con numeración y explicaciones detalladas de sus funciones.	2			
4. Ejecución	Se incluyen ejecuciones/pruebas del código fuente explicadas gradualmente.	2			
5. Pregunta	Se responde con completitud a la pregunta formulada en la tarea. (El profesor puede preguntar para refrendar calificación).	2			
6. Fechas	Las fechas de modificación del código fuente estan dentro de los plazos de fecha de entrega establecidos.	2			
7. Ortografía	El documento no muestra errores ortográficos.	2			
8. Madurez	El Informe muestra de manera general una evolución de la madurez del código fuente, explicaciones puntuales pero precisas y un acabado impecable. (El profesor puede preguntar para refrendar calificación).	4			
Total		20			

12. Referencias

- https://developer.mozilla.org/en-US/docs/Learn/Server-side/Django/Tutorial_local_library_website
- https://github.com/mdn/django-locallibrary-tutorial
- https://github.com/rescobedoq/pw2/tree/main/labs/lab05
- William S. Vincent. (2022). Django for Beginners: Build websites with Python. Django 4.0. leanpub.com. [URL]
- https://docs.djangoproject.com/en/4.1/ref/models/fields/
- https://docs.djangoproject.com/en/4.0/topics/db/examples/many_to_many/
- https://docs.djangoproject.com/en/4.0/topics/db/examples/many_to_one/
- https://blog.hackajob.co/djangos-new-database-constraints/
- https://stackoverflow.com/questions/3330435/is-there-an-sqlite-equivalent-to-mysqls-describe-ta
- https://docs.djangoproject.com/en/4.1/ref/validators/#how-validators-are-run
- https://docs.djangoproject.com/en/4.1/ref/models/instances/
- https://www.youtube.com/watch?v=rHux0gMZ3Eg
- https://www.youtube.com/watch?v=OTmQOjsl0eg
- https://tex.stackexchange.com/questions/34580/escape-character-in-latex
- https://www.sqlitetutorial.net/sqlite-show-tables/
- https://www.wplogout.com/export-database-diagrams-erd-from-django/